

Learning Velocity from IMU Signals: AxisTilted2 at the IROS 2026 IMU Odometry Challenge

Sanjayan Sreekala and Chinmayan Pradeep
Independent Researchers

san@sankala.me chinpradk@gmail.com

Abstract—The IROS 2026 IMU Odometry Challenge asks teams to estimate velocity from inertial sensors across cars, quadrupeds, drones and handheld devices using one shared model. Trained from scratch on the released data, our model combines a convolutional encoder, frequency features and a bidirectional recurrent network to predict velocity. Learned sensor corrections feed an inertial smoother that reconciles these predictions with motion equations. Three choices helped in development: training with different context lengths, using sensor corrections inside the smoother, and supervising complete scored paths. Path supervision primarily reduced trajectory drift rather than velocity error. A diagnostic on an earlier model also revealed why better sensor estimates need not improve motion estimates: the smoother could cancel an acceleration correction by changing its estimated bias, leaving velocity almost unchanged. The final submission scored 0.27765 on the public leaderboard and 0.21556 in the official all-89-sequence evaluation. Drone velocity errors remain the main weakness. Code and trained weights are publicly available.

I. INTRODUCTION

An inertial measurement unit (IMU) contains accelerometers and gyroscopes. These measure specific force, including the effect of gravity, and angular velocity. They are compact sensors used in robots, vehicles and handheld devices. Estimating motion from these signals alone is difficult: an IMU does not directly measure velocity, and small errors accumulate when its measurements are integrated over time.

The IROS 2026 IMU Odometry Challenge [4] asks participants to predict one three-dimensional velocity vector per second from six-axis IMU measurements sampled at 200 Hz. The target is mean velocity in the *body frame*: the three axes attached to the moving sensor, rather than fixed world axes. One shared model must handle four types of motion: wheeled cars, legged quadrupeds, aerial drones and human-carried devices. At inference, our model receives only the raw IMU signals and public recording/window boundaries. It receives no reference position, orientation, velocity or platform label.

Our starting idea was to combine two sources of information. A neural network can learn recurring motion patterns, such as the relationship between vibration and movement. Inertial equations can connect nearby velocity estimates through measured acceleration and rotation. The equations alone cannot recover an arbitrary initial velocity, while a learned predictor can make errors that are inconsistent over time. We therefore train a system that learns both a velocity estimate and how strongly to adjust it using those equations.

We ask whether varying the training context improves predictions, whether better sensor corrections improve velocity,

and whether training on complete scored paths reduces drift. In one diagnostic, estimated bias cancelled an acceleration correction, leaving velocity almost unchanged despite better sensor consistency. The strongest evidence comes from matched comparisons, where competing changes start from the same model and receive the same extra training. Better training fit also did not consistently improve predictions on held-out recordings.

This approach builds on learned inertial estimation. Tartan-IMU [1] studies shared representations across different platforms. TLIO [2] combines learned displacement estimates with a filter; AI-IMU [7], KalmanNet [8] and Backprop KF [3] also combine learning with state estimation. Our contribution is the submitted shared model and the experimental evidence about which combinations helped in this challenge, rather than the general idea of combining neural networks and filtering.

A. What the Challenge Measures

The challenge evaluates both velocity and the path obtained by integrating it. Absolute Velocity Error (AVE, in m/s) is the mean per-window Euclidean distance between predicted and reference velocity vectors. Absolute trajectory error over 20-meter segments (ATE₂₀, in meters) measures position error after aligning the predicted and reference paths. Lower is better for both. The ranking score combines them as

$$S = 0.6 \frac{\text{AVE}}{0.7356384388} + 0.4 \frac{\text{ATE}_{20}}{3.1160277267}. \quad (1)$$

Errors are averaged over trajectories within each platform, then equally across the four platforms. The constants are the challenge’s normalization factors. Relative trajectory error (RTE) measures five-second displacement error and is reported separately; it does not affect ranking. The combined score can therefore improve through reduced path drift even when velocity error does not improve.

II. METHOD

A. Predict Motion, Then Enforce Consistency

Figure 2 shows the three parts of the model. First, a neural predictor estimates velocity from the IMU. Second, a separately trained sensor network estimates corrections to the sensor signals and the direction of gravity. Third, a smoother combines the neural predictions with inertial equations. It adjusts estimates jointly over a time interval, using both

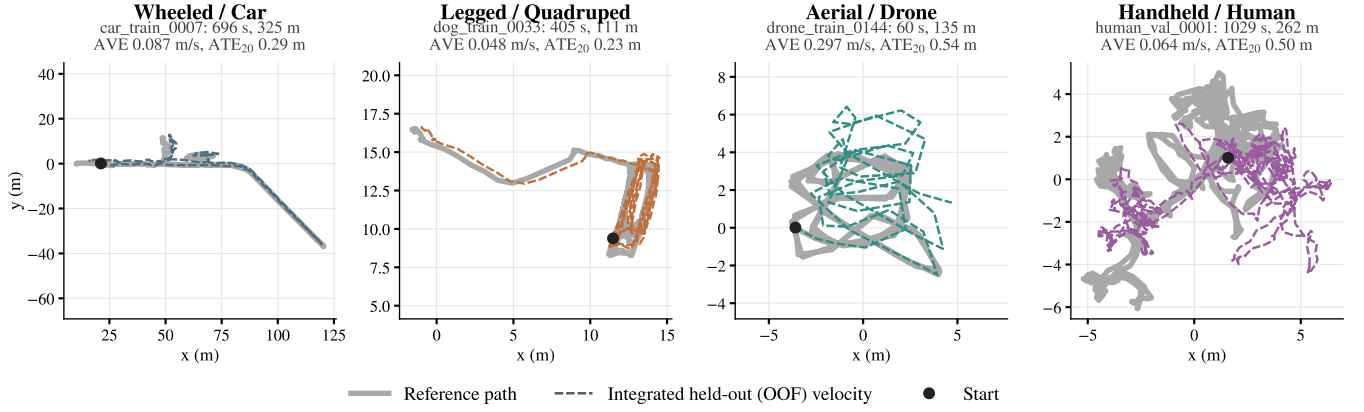


Fig. 1. Full-recording trajectories from held-out predictions. Each platform is represented by a recording at the median rank of out-of-fold ATE₂₀, predicted by a fold model that did not train on it. Velocities are rotated using reference mid-window attitude, as in the official metric; attitude is not estimated here. Paths start at the reference position and retain accumulated drift over the entire recording (up to 17 minutes), without subsequent alignment.

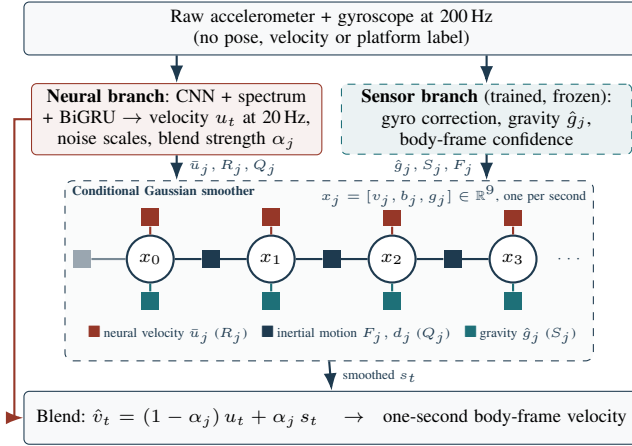


Fig. 2. Shared model architecture. The neural branch learns motion patterns; the frozen sensor branch supplies gravity and corrected rotation. In the smoother, circles denote one-second states and squares denote costs for neural velocity, gravity, inertial motion and the initial-state prior. A block-tridiagonal linear solve yields the states (Appendix F).

earlier and later measurements. The model learns how much of this adjustment to retain.

The neural predictor combines short and long time scales. A convolutional neural network (CNN) extracts local patterns. Summary statistics and a spectrogram—a representation of how signal frequencies change over time—provide complementary features. A bidirectional gated recurrent unit (BiGRU) combines these features with past and future context. A decoder produces velocity at 20 Hz, later averaged to one-second predictions.

The decoder has one shared output and four small correction heads. An input-derived gate assigns a weight to each correction. Platform labels supervise this gate during training, but are not needed at inference. These heads are components of one shared network, not separately trained models selected by a supplied platform identity.

B. How Sensor Corrections Enter the Smoother

The sensor network predicts corrections to gyroscope rotation, scale, bias and timing, as well as gravity direction and a confidence in how well the body-frame motion equations apply. It is trained using the released reference geometry and then frozen before velocity training. Its outputs are learned effective corrections; they are not independent measurements of the hardware’s calibration.

The smoother estimates three quantities at each one-second boundary: velocity, accelerometer bias and gravity, each with three components. Bias represents an offset in the acceleration measurement. The solver balances agreement with neural velocity, agreement with predicted gravity, and consistency between successive states under inertial equations. The neural model supplies weights for these competing terms. Initial velocity is only weakly constrained, so the method does not assume that a recording starts at rest.

This is a conditional Gaussian smoother: once the learned corrections and weights are given, its states are found by solving a quadratic problem. Unlike a forward-only filter, it uses future measurements within the context. Its velocity is blended with the direct neural prediction using a learned strength and body-frame confidence. Both estimates come from the same IMU, so the weights should be understood as learned blending parameters, not a guarantee of calibrated statistical uncertainty. Appendix F gives the equations and architecture dimensions.

C. Data and Training

Table I summarizes the data used. The final model is trained from scratch on all 475 released labelled trajectories: 395 from the original training split and 80 from the original validation split. A trajectory is one recording. The submission uses no external training data or pretrained weights. Earlier team diagnostics involving NormWear were excluded following the organizer’s external-data ruling [5].

Training samples the four platforms equally. Context windows are 64 seconds long, with half shortened to 8, 16 or 32 seconds to expose the model to different amounts of context.

TABLE I
DATA USED BY THE SUBMITTED MODEL

Resource	Use
Released train: 395 trajectories	Model selection and final training
Released validation: 80	Model selection and final training
Anonymous test: 89	Prediction from IMU only
External data / pretrained weights	None in submitted pipeline
Synthetic / pseudo-labelled data	None; only rotations of training examples

Small rotations, applied consistently to inputs and targets, augment the training data. Neural inputs are normalized using only the training partition. Targets are averages of the released body-frame velocities, and windows never cross recording boundaries.

The selected schedule fits the sensor network for 4,000 updates, the velocity model for 16,000, and then continues velocity training for 4,000 updates with complete-segment supervision. Ordinary training penalizes velocity error on short crops and displacement error over several time horizons. Complete-segment supervision adds a loss on whole 20-meter scoring segments. It runs the same overlapping contexts used at inference, joins their predictions, and penalizes the resulting path error. This allows training to address drift across context boundaries. Reference orientations and positions are used only to calculate training losses.

The submitted checkpoint uses an exponential moving average (EMA) of weights during training. It is one final fit on all released labels; no independently trained models or cross-validation models are averaged. Optimization settings and the exact segment loss are in Appendix F.

D. Inference and Reproduction

Inference processes 64-second contexts with a 32-second stride. Predictions in overlaps receive smooth weights before being averaged, and state is reset between trajectories. There is no test-time fitting. The use of future context makes this an offline method. Table II reports its throughput.

Two runs on the recorded RTX 3090 setup reproduced all 30,644 submitted rows exactly. A fresh RTX 4090 install also produced two matching runs. Between GPUs, the mean velocity-vector difference was 0.000421 m/s and the maximum absolute component difference was 0.014917 m/s. Cross-hardware byte identity is not claimed. Appendix B provides both reference hashes and environments.

III. DEVELOPMENT EXPERIMENTS

A. How We Compared Changes

Early experiments used two held-out development sets: a motion-family split designed to keep related recordings together, and the original released validation split. A held-out recording is not used to fit the model being evaluated. These comparisons test individual changes against their own controls; scores from different stages are not directly comparable.

For the final training choice, we split all 475 labelled trajectories into three groups, keeping 14 known related

TABLE II
RECORDED RUNTIME AND REPRODUCIBILITY

Property	Value
Training / inference GPU	NVIDIA RTX 3090, 24 GB
Inference time	9.79 s for 30,644 one-second windows
Effective output throughput	Approximately 3,130 windows/s
Reserved CUDA memory	0.52 GB
Context / stride	64 s / 32 s; non-causal
Weights	One final EMA model, frozen shared sensor
Parameters	3,549,642 total; 86,610 in frozen sensor
Training wall time	Not recorded

recording pairs together. Each model trained on two groups and predicted the third. Pooling the predictions gives a held-out velocity estimate for every one-second window in every recording. We call this out-of-fold (OOF) evaluation. The groups contained 159, 158 and 158 trajectories. Sensor fitting and normalization were repeated using only the training side of each split. We had already consulted these data during development, so this comparison is useful for choosing a recipe but is not an untouched test of generalization.

B. Did Changing the Training Context Help?

An early audit showed a large gap between errors on training and held-out recordings, particularly for drones (Fig. 3). This motivated us to test changes that might improve predictions on other recordings, rather than training fit alone.

One useful change was to vary how much context the model saw during training. Mixing shorter windows with 64-second windows reduced the development score from 0.19865 to 0.18719, a 5.77% improvement. On a separate held-out fold, it reduced the score from 0.17832 to 0.17304, a 2.96% improvement. Most of that second gain came from drones; cars and handheld recordings did not improve. We retained mixed context lengths in the submitted model.

These comparisons used the same number of training updates, but shorter windows meant fewer total seconds of input. They support the tested training recipe without isolating context length from data exposure. Changing the neural architecture was less useful: the tested Transformer scored 0.23072 against the 0.19865 development control, and later tests of faster recurrent updates and whole-recording memory gave no consistent gain.

C. Did Better Sensor Information Improve Velocity?

Adding frequency information helped in a separate development study: a learned spectrogram reduced the score by 3.33% on the motion-family holdout and 2.73% on the original validation set. We then tested two ways of using learned sensor corrections. One supplied them to the neural decoder as extra features. The other used them inside the smoother, changing the inertial constraints between velocity estimates.

The matched comparison used the same starting model and training budget. Lower scores are better:

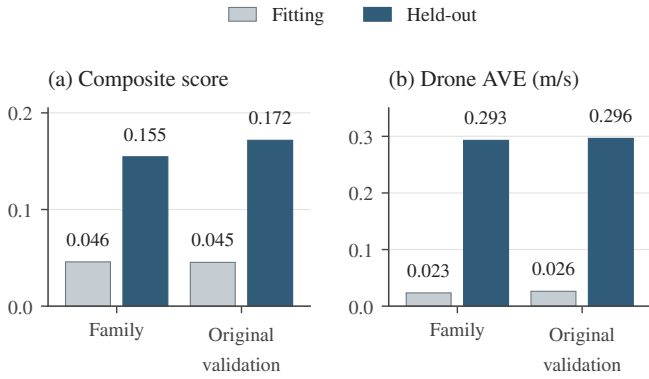


Fig. 3. Fitting and held-out errors. Each pair comes from the same training run in an earlier reference audit, with different fitting and held-out populations. These are separate from the final three-fold experiment.

	Motion-family holdout	Original validation
Continued training (control)	0.19338	0.19908
Corrections as extra features	0.19341	0.19910
Corrections inside smoother	0.17855	0.19277

The extra features had almost no effect. Using the corrections in the smoother reduced scores by 7.67% and 3.17%, respectively. Mean velocity and trajectory errors improved for all four platforms on both sets. We therefore retained this way of combining sensor information with neural predictions.

A later candidate added acceleration corrections that improved a sensor-consistency diagnostic. Its velocity score, however, remained almost identical to its continued-training control and worse than the retained model (Appendix Table VII). Section V examines how the smoother’s bias estimate can absorb such a correction.

D. Did Training on Complete Paths Reduce Drift?

Short windows are convenient for training, but the challenge also scores paths that can cross several windows. We tested whether supervising complete scoring segments would help the model reduce this accumulated error. In each of the three folds, four continuations started from the same base model and received 4,000 further updates: ordinary training (control), segment supervision, speed augmentation by rescaling time, and both changes together.

Segment supervision was best (Table VI). It improved all three folds by 0.925%, 1.192% and 2.473%, and reduced the pooled score from 0.164142 to 0.161758, a 1.45% improvement. Average trajectory error improved 3.64%, while average velocity error worsened 0.25%. Adding speed augmentation, alone or with segments, did not beat the control. We therefore selected segment supervision without speed augmentation, then trained one fresh model on all released labels.

The public score improved from 0.28427 to 0.27765 between the earlier and final submitted pipelines. That change also included a different sensor-fitting scope and training schedule. The public improvement therefore cannot be at-

TABLE III
OFFICIAL OVERALL RESULTS

Method	Score	ATE ₂₀	AVE	RTE
Ours	0.21556	0.56008	0.17614	0.82921

All 89 test sequences, evaluated using the selected CSV. ATE₂₀ and RTE are in meters; AVE is in m/s. The final private leaderboard score is unavailable.

TABLE IV
OFFICIAL RESULTS BY PLATFORM

Platform	ATE ₂₀	AVE	RTE
Wheeled / Car	0.36293	0.07003	0.27881
Handheld / Human	0.54497	0.04514	0.27870
Legged / Quadruped	0.33028	0.07432	0.35831
Aerial / Drone	1.00216	0.51505	2.40103
Macro average	0.56008	0.17614	0.82921

All 89 test sequences. ATE₂₀ and RTE are in meters; AVE is in m/s.

tributed entirely to segment supervision; the matched three-fold experiment isolates the evidence for that change.

IV. EXPERIMENTS

A. Overall and Per-Platform Performance

The selected submission scored 0.27765 on the public leaderboard, where AxisTilted2 was sixth as of September 17, 2026 (September 18 UTC). This is a dated public position, not a final private ranking. Separately, the official scoring service [6] evaluated the exact submitted CSV over all 89 test sequences and returned 0.21556 (Table III). The two scores cover different populations. The service receipt confirms CSV MD5 62c722045add864387c1b01c83695a1b.

The platform breakdown makes the remaining weakness clear (Table IV). Drones have an average velocity error of 0.51505 m/s and five-second relative trajectory error of 2.40103 m. They account for 63.6% of the composite error despite equal platform weighting; drone velocity alone contributes 48.7% (Fig. 4). The other platforms’ velocity errors range from 0.04514 to 0.07432 m/s. Appendix Table V gives every official sequence result, including the difficult drone sequences 56 and 29. These numbers are scoring-service identifiers, not matched source-recording identities.

B. Where the Chosen Training Change Still Fails

Figure 1 shows complete integrated paths for representative held-out recordings. We use the labelled development recordings to examine behavior in more detail. The benefit of segment training is uneven: 252 OOF trajectories improve and 223 worsen in their contribution to the score (Fig. 5). On one quadruped recording, dog_val_0010, trajectory error falls from 3.01551 to 1.66687 m (44.72%), while velocity error worsens 4.66%. That recording alone accounts for 35.6% of the net pooled gain.

Figure 6 shows a different problem in three drone recordings. The model maintains an incorrect velocity offset or misses a change in direction. Segment training

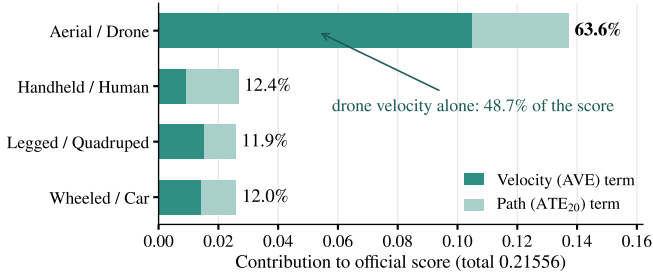


Fig. 4. Official score decomposition. Each bar shows one platform’s contribution, split into velocity and path terms. Values are computed from Table IV and sum to 0.21556.

TABLE VI
THREE-FOLD OUT-OF-FOLD COMPARISON

Recipe	Pooled	Fold 0	Fold 1	Fold 2
Base	.164771	.188753	.158704	.146430
Control	.164142	.187759	.158343	.145905
Segments	.161758	.186023	.156456	.142297
Speed	.166785	.193262	.159334	.147439
Both	.164160	.190803	.157796	.143489

Each continuation receives the same extra training from the fold’s base. The final submission is one fresh fit on all released labels.

barely changes these predictions. The final OOF velocity errors are 2.58859 and 2.43655 m/s on the related recordings `drone_train_0023` and `0024`, and 0.69430 m/s on `drone_val_0002`. The improvement in path error therefore does not mean that the model has learned to recover difficult signed velocities.

V. INSIGHTS AND LESSONS LEARNED

A. Interpreting Sensor Corrections Together with Bias

The acceleration-correction diagnostic shows how two parts of a learned estimator can explain the same measurement. Our smoother estimates accelerometer bias as well as velocity. If corrected acceleration enters the dynamics as $a + c - b$, where c is a learned correction and b is estimated bias, increasing the bias by c cancels the correction. Velocity can therefore stay nearly unchanged even when the corrected sensor signals become more consistent.

We measured this behavior on the first 64 seconds of 48 drone recordings selected for body-frame consistency in an earlier model. The bias change absorbed 98.5–99.3% of the added acceleration correction across the training and two held-out sets. Mean velocity changed by only 0.00074–0.00128 m/s. Mildly restricting bias helped one holdout but barely changed the other; stronger restrictions increased error.

A sensor correction should therefore be evaluated through its effect on both estimated bias and final velocity. Supervision or motion diversity that separates these two explanations may help the correction improve velocity.

B. Velocity and Trajectory Metrics Reveal Different Gains

Segment supervision improved integrated path accuracy while average velocity error changed little. One quadruped

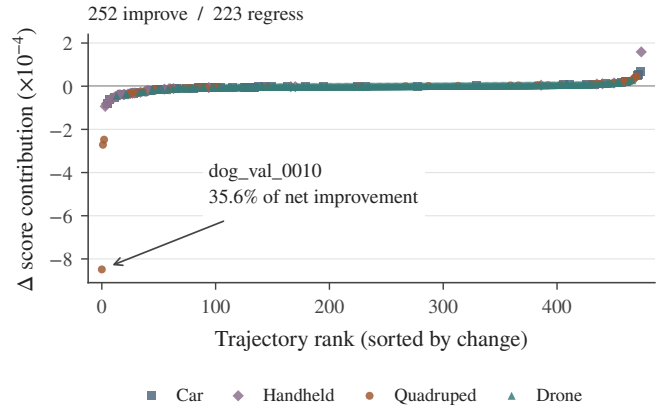


Fig. 5. Changes in score contribution across 475 held-out recordings. Points are sorted by segment-training contribution minus control contribution; negative values indicate improvement. The largest improvement supplies 35.6% of the net gain.

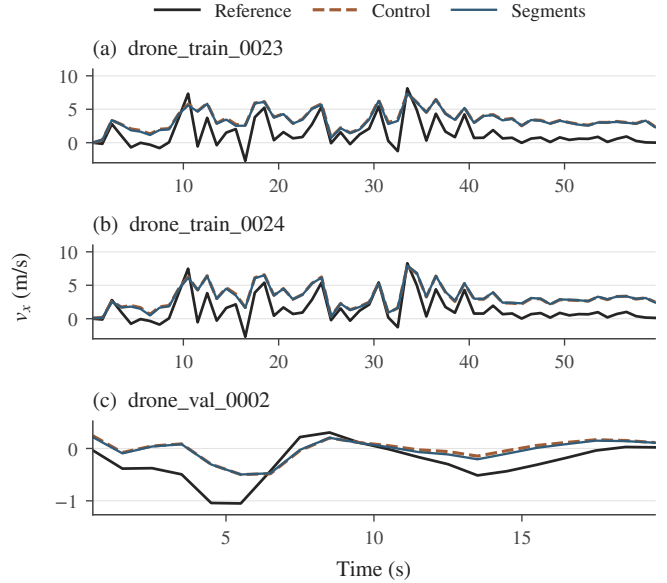


Fig. 6. Velocity predictions on held-out drone recordings. The plots show one-second velocity along the body x -axis. Both training recipes miss sustained offsets or negative motion; segment-trained predictions nearly coincide with the control.

recording contributed a large share of the gain, while the drone traces retained sustained velocity offsets.

A selector given the correct answers could choose among eight earlier predictions and reduce error by roughly 29% on the motion-family holdout and 32% on original validation. A learned selector using available inputs improved neither score by more than 1.3%; averaging increased error. These prototypes informed development; the submission uses the single shared model described in Section II.

C. Scope of the Evidence

The shared model operates across all four represented platforms, with its largest remaining velocity errors on drones. Evidence for transfer is limited to held-out recordings from these four platforms. The grouped-fold comparisons

use one training run per configuration in each fold on development data consulted during model design; historical public scores also informed development. Their strongest conclusion concerns the tested training choices, particularly segment supervision and speed augmentation. Appendix Table VII provides the controls and outcomes for all 37 studies.

D. Feedback to the Organizers

The released labels and official per-sequence scoring service supported detailed analysis of individual recordings. For future editions, identifying causal versus offline methods would clarify their use of temporal context. Recording-family identifiers would support consistent grouped splits; our final folds kept 14 related pairs together.

A worked example linking window-mean velocity targets to trajectory integration would help explain attitude, integration duration and alignment. Reporting velocity and trajectory metrics alongside composite scores, with numerical reproduction tolerances, would support comparisons across methods and hardware.

VI. CONCLUSION

AxisTilted2 combines a shared neural velocity predictor with learned sensor corrections and inertial smoothing. Varying training context and using sensor corrections inside the smoother improved development scores. Training on complete scoring segments added a smaller gain, chiefly in path accuracy. The final submission reached a public score of 0.27765, but sustained drone velocity errors remain

unresolved. For further work, the central questions are how to learn motion patterns that hold across recordings and how to make improved sensor estimates change velocity usefully.

ACKNOWLEDGMENT

We thank the challenge organizers for the released data, evaluation tools and public rule clarifications. The authors are independent researchers.

REFERENCES

- [1] S. Zhao, S. Zhou, R. Blanchard, Y. Qiu, W. Wang and S. Scherer, “Tartan IMU: A Light Foundation Model for Inertial Positioning in Robotics,” in *CVPR*, 2025, pp. 22520–22529.
- [2] W. Liu et al., “TLIO: Tight Learned Inertial Odometry,” *IEEE Robotics and Automation Letters*, vol. 5, no. 4, pp. 5653–5660, 2020. <https://arxiv.org/abs/2007.01867>.
- [3] T. Haarnoja, A. Ajay, S. Levine and P. Abbeel, “Backprop KF: Learning Discriminative Deterministic State Estimators,” in *NeurIPS*, vol. 29, 2016.
- [4] CMU AirLab and Super Odometry Group, “IMU Odometry Challenge: setup and report requirements,” accessed September 17, 2026. <https://superodometry.com/imuchallenge/setup/>.
- [5] TartanIMU organizers, external-data clarification, discussion 737628, August 30, 2026. <https://www.kaggle.com/competitions/tartan-imu-challenge-iros2026/discussion/737628>.
- [6] Tartan-IMU, “IMU Odometry Challenge per-sequence scoring,” exact-CSV evaluation, September 18 UTC, 2026. https://huggingface.co/spaces/Tartan-IMU/imu_odometry_challenge_scoring.
- [7] M. Brossard, A. Barrau and S. Bonnabel, “AI-IMU Dead-Reckoning,” 2019. <https://arxiv.org/abs/1904.06064>.
- [8] G. Revach et al., “KalmanNet: Neural Network Aided Kalman Filtering for Partially Known Dynamics,” 2021. <https://arxiv.org/abs/2107.10043>.

APPENDIX

A. Per-Sequence Results

TABLE V
OFFICIAL RESULTS FOR ALL 89 TEST SEQUENCES

ID	ATE ₂₀	AVE	RTE	ID	ATE ₂₀	AVE	RTE	ID	ATE ₂₀	AVE	RTE
1	0.99472	0.45088	2.26425	31	0.91336	0.45196	1.99605	61	1.19335	0.27650	1.25659
2	0.65109	0.30819	1.39709	32	0.36707	0.09135	0.27439	62	0.07666	0.01658	0.05108
3	0.59504	0.27297	1.05808	33	0.30208	0.05256	0.23137	63	0.39720	0.05359	0.28840
4	0.39184	0.18669	0.66863	34	0.70833	0.21561	0.98082	64	0.45278	0.04627	0.25282
5	0.72121	0.28774	1.21961	35	0.96488	0.21027	1.12983	65	0.06994	0.03233	0.08714
6	0.68909	0.08882	0.75275	36	1.04997	0.49500	2.18199	66	1.49547	0.24522	1.71309
7	1.03168	0.29352	1.28853	37	1.93160	1.49026	9.00090	67	0.20165	0.03191	0.20616
8	0.28713	0.03243	0.18316	38	1.85365	1.48067	8.82287	68	0.59704	0.04819	0.24027
9	0.47882	0.22131	0.62621	39	0.37967	0.17312	0.57472	69	0.75363	0.06466	0.44992
10	0.96268	0.08644	0.71853	40	0.86450	0.41991	2.00311	70	0.50885	0.07133	0.30546
11	1.06914	0.72672	1.79068	41	1.12526	0.57294	2.34945	71	0.15639	0.05826	0.17392
12	0.15261	0.23598	1.04566	42	0.38439	0.07789	0.31397	72	0.91517	0.47745	1.37433
13	0.20712	0.03649	0.15455	43	0.72610	0.30062	1.14304	73	0.52593	0.11987	0.45803
14	0.68093	0.32818	1.36176	44	0.24426	0.05272	0.32421	74	0.47407	0.22972	0.80218
15	1.55935	0.09030	0.61446	45	0.11031	0.05633	0.14315	75	0.63437	0.30891	1.24329
16	0.14526	0.03581	0.23394	46	0.28310	0.04418	0.22781	76	0.33692	0.07496	0.22545
17	1.21241	0.45881	2.52286	47	0.98725	0.36853	1.42404	77	1.18374	0.60190	2.14335
18	1.15385	0.40297	2.06281	48	0.22923	0.01403	0.10625	78	0.14695	0.02672	0.09486
19	0.72549	0.35078	1.57433	49	0.74550	0.29503	1.18385	79	0.38050	0.18191	0.75717
20	0.13077	0.05313	0.37574	50	0.55148	0.19050	0.77669	80	0.43046	0.20189	0.99426
21	0.71576	0.23694	1.09999	51	0.10416	0.04768	0.15045	81	0.39264	0.12002	0.37659
22	0.65626	0.33907	1.33784	52	0.12682	0.05669	0.17770	82	2.41464	0.29486	1.82614
23	1.01780	0.39543	1.96857	53	0.40761	0.28082	0.54309	83	0.47315	0.07490	0.36981
24	0.49575	0.03402	0.18965	54	0.19583	0.06589	0.23139	84	0.45106	0.04643	0.28972
25	0.21484	0.02699	0.10392	55	0.31569	0.25113	1.16984	85	0.08632	0.03572	0.11201
26	1.00989	0.11190	0.46839	56	4.36859	3.29140	16.69667	86	0.32814	0.02502	0.15647
27	0.49542	0.04467	0.25617	57	1.09687	0.50486	2.47850	87	0.43271	0.18345	0.76721
28	0.32178	0.16025	0.66422	58	0.22317	0.05009	0.25679	88	1.11771	0.52994	2.26397
29	3.95025	3.22689	15.21614	59	0.35572	0.08189	0.32277	89	0.12806	0.02646	0.09568
30	0.81726	0.29564	1.53814	60	0.15701	0.06967	0.23704				

Exact CSV MD5: 62c722045add864387c1b01c83695a1b; scored September 18 UTC, 2026. IDs are issued by the scoring service. ATE₂₀ and five-second RTE are in meters; AVE is in m/s.

B. Reproducibility and Submission Artifacts

The selected artifact is Kaggle submission **56287620**, team **AxisTilted2**, with 30,644 prediction rows. The public release contains the model, exact prediction CSV, frozen inference code and training source. The hashes below identify these artifacts.

CSV MD5

62c722045add864387c1b01c83695a1b

CSV SHA-256

529ebc972daed3cea5de317096769436
a87a652af35819383bcb1164388dc916

RTX 4090 reference CSV SHA-256

35683cd11a39f2ddf91390c063518e3
e6fbcb7053530df80e1f7623710eb14e0

Model SHA-256

0caa21fed31c77b5ed11f258d4945906
4e5cf6e6a72a318f3e52e28718fcd1f9

Frozen training source SHA-256

94e150da0446e957a67dcefa4ca3a872
35e0e1afe3d26ad146f98f6d750c09a5

Public artifacts: [AxisTilted2/tartanimu-iros2026](#); [exact prediction CSV](#). The checkpoint embeds inference configuration and fitting provenance. Recorded runtime: Python 3.12, PyTorch 2.10.0+cu128, NumPy 2.5.3, pandas 3.0.5, SciPy 1.18.1, PyYAML 6.0.3. The fresh RTX 4090 environment used Python 3.12.2, the pinned dependencies and cuDNN 9.10.2; its receipt records 8.31 seconds and 0.52 GB reserved memory. Window IDs and order match. Disabling TF32 did not remove the difference; BF16 and hardware-dependent cuDNN kernels are suspected sources. Cross-hardware byte identity is not guaranteed. The 4090 output was not scored. The grouped-fold construction seed is 2026091601. Velocity bases use seeds $20265441 + i$ for folds $i = 0, 1, 2$; continuations add 100 and sensor fitting adds 9001 to each base seed. The all-label base uses 20265441, sensor 20274442, and continuation 20265541. Reported OOF values pool held-out predictions for all windows across the 475 trajectories. The final public result is a single all-label fit, not an average over seeds; run-to-run spread was not measured.

On macOS CPU, mean/max absolute component differences from the submitted CSV were 0.000381/0.071294 m/s. Both GPU references are public; all competition scores refer to the original RTX 3090 CSV. Full training replay additionally requires the data-preparation manifests/tables.

C. Required Compliance Declaration

- 1) **Single shared model?** Yes. A single shared velocity model with jointly trained internal residual heads generates the predictions. Its shared sensor network is separately fitted on released labels and frozen inside the pipeline. There are no independently trained per-platform velocity models selected externally.
- 2) **Routing or specialization at inference?** Yes. A four-way input-derived soft gate mixes internal

residual velocity heads. Platform labels supervise that gate during fitting only. A learned body-convention confidence also gates calibration feedback and smoother blending. Neither gate uses supplied test platform labels or recovered source identities.

- 3) **Ensembling, weight averaging or TTA?** Training uses EMA with decay 0.995, and that final EMA state supplies one checkpoint. Overlapping predictions from the same checkpoint are smoothly averaged (64-second context, stride 32). There is no independently trained model/fold ensemble, checkpoint soup, rotational TTA or test-time optimization.
- 4) **Leakage avoidance and selection?** No test labels, external source matching or test ground-truth geometry are used in the submitted pipeline. Fold-specific normalization, sensor fitting and velocity initialization use only their fitting trajectories, with known related pairs grouped together. All labels in the final refit are released train/validation labels. The final continuation recipe is chosen from grouped OOF comparisons before its all-label fit; historical public scores informed the broader development process, so public performance is not a blind estimate. The folds themselves also reuse previously consulted labelled data. NormWear diagnostic entries and external parameters are excluded from the final pipeline. The official service tables use only the exact submitted CSV.
- 5) **Development versus submitted system?** Development compares several models and four continuation conditions across three folds. Submission uses only the segment-supervised recipe, refitted from scratch on all 475 labelled trajectories with a fresh all-label sensor. No speed augmentation, geometry teacher or fold averaging is retained. Training has access to released geometry and platform supervision; inference receives only raw IMU and public boundaries.

D. Final Predictions

The exact CSV has columns `window_id`, `vx`, `vy`, `vz`, 30,644 unique IDs and mean body-frame velocities in m/s. No extra gain or smoothing follows context blending. The [public prediction CSV](#) preserves the submitted bytes. Code and weights are designated Apache-2.0.

E. Experiment and Decision Ledger

This ledger summarizes the development studies and the evidence behind model selection. Decisions reflect the stage of development at the time of each study. F/O denotes motion-family/original-validation holdouts, not public/private test. Development, prefix diagnostics and final pooled OOF use different populations and fitting scopes. Rows must not be ranked as one leaderboard. Oracles and unsubmitted prototypes remain explicitly separate.

TABLE VII: Complete experiment and decision ledger

Study / arms	Results (lower is better unless stated)	Decision / limit
Initial source and account assessment: reuse the account's best public model?	NormWear provenance was explicitly rejected by organizers for final consideration.	Exclude those diagnostic submissions regardless of public score. Build a shared released-data-only model.
First training run: conventional competitive baseline	Shared CNN/BiGRU, 46 statistics, balanced sampling, augmentation, dense/integrated losses, EMA; development .194628. Rotation TTA did not pass selection.	Retain baseline. The all-label fit has no independent labelled validation score.
Initial bounded sweep: longer schedule, integration strength, rotations and inference context	Completed first uploaded eligible pipeline; public .30815. Detailed diagnosis records the exact evaluated artifact.	Local candidate selection before upload; later public gap required diagnosis, not another blind parameter grid.
Official report and 51-candidate frozen screen	Old CSV full-test .24051 across 89 trajectories; drones 63.7% of score. Cross-fitted correction 2.7%; four-model mean+correction 5% locally; Gaussian smoothing harmful (.220/.337/.460 at .5/1/2 s); feature-only boosted trees .602.	Report is for old CSV and a different set from public leaderboard. Calibration/ensemble gains were exploratory; no eligible final ensemble established. Drop harmful smoothing and weak feature-only model.
Same frozen screen: routing, time shifts, sharpening, deadbands, longer contexts and EMA/raw mixtures	Routing almost unchanged; shifts/sharpening small, 256/512 s .188617/.188611 vs .189770; no stable mixture gain.	Useful cheap screening, not evidence of an indexing bug or a breakthrough. Later independent holdout was needed for calibration.
Physics identifiability, first principles	Long-horizon label/IMU consistency deteriorates even with fitted nuisance states. Constant velocity level is unobserved by pure IMU. Fourteen related motion pairs found; six crossed an early auxiliary split.	Family holdouts repaired; short reliable constraints prioritized. Label-fitted physics and perfect speed/direction are diagnostics, not inference methods; source identity guesses are excluded. Removing affected records did not erase the continuation gain.
Seven-arm research screen: crop coverage, attention, normalization, auxiliary features/losses	Development control .198650; mixed contexts .187195; time-local norm .238229; Transformer .230724; auxiliary speed bins .195327; split-frequency encoder .196438; cropped segment loss .195624. Fresh-fold control/mixed .178319/.173036.	Retain mixed context; strongest tested transfer gain. Other small gains not promoted; attention and normalization lost in this bounded recipe. This was one extra neural holdout, not full K-fold.
KISS-inspired/hybrid comparison	Reference .187195; motion-balanced .191575; relative auxiliaries .188319; raw physics .191284; label geometry .187013. Fixed local solvers added only tiny gains.	No arm clears .001 promotion margin; skip conditional combination/new fold. Auxiliary CUDA RNG differed from reference: a recorded seed confound, not leakage. This does not establish a general rejection of KISS-IMU or physics.
Finishing controls: context, cross-fitted gain, continuation	64 s retained; gain calibration helped development but hurt separate holdout. Safe 4k continuation .173062 to .162064. Resulting public .30588.	Reject calibration, keep coverage/continuation evidence. Public gain was small, triggering first-principles reassessment. Continuation changed coverage and duration together.
Frame augmentation/consistency	F/O control .200681/.208526; broad rotations .200188/.216071; consistency .198514/.212895.	Neither improves both scopes. Keep failure/family diagnostics; do not promote augmentation on one favorable number.
Representation screen	F/O spectral .194005/.202825; interactions .203648/.211242; preintegration .198026/.210156; local joint .258914/.271651; curriculum .298429/.318739. Reference .200681/.208526.	Retain spectral ingredient (3.33%/2.73%), below 5% breakthrough gate. Others fail consistent transfer. Preintegration implementation was numerically checked, but correct math did not imply better predictions.
Convention-aware training	Corrected augmentation .203416/.215054; internal convention head .199939/.219580; balanced sampling .209256/.229567.	All fail two-set improvement. Heterogeneous-frame evidence remains informative, but changed annotation/augmentation/gating did not solve transfer.

Continued on next page

Table VII continued

Study / arms	Results	Decision / limit
Learned local transport	Auxiliary .202164/.215961; 8-second smoother .194606/.208616; 16-second .192658/.211244.	Better than some batch-internal controls is insufficient: neither beats retained spectral model on both sets. Promotion requires the retained reference.
Gravity state	Auxiliary .199538/.215400; feedback .201081/.209912; gravity smoother .184737/.205117. Smoother-off ablation .203185/.208576.	Fusion materially helps some drones but misses two-set margin and spectral official score; human AVE/ATE tradeoff. Supports calibration/uncertainty investigation rather than simple gravity features.
Body-only robustness	Views .197965/.211508; noise .201903/.212758; views+balance .201954/.219269.	No two-set gain; stop these recipes. More hard-example sampling is not justified merely by large already-fitted errors.
Separate sensor calibration	Temporal sensor improves body-drone rotation discrepancy raw .0605/.0677 to .0260/.0160 rad/s; gravity 3.72/1.83 to 1.63/.99 m/s ² . Both affine/temporal meet sensor gate.	Select temporal sensor, then test complete velocity pipeline. Sensor scores alone are not competition velocity scores.
Calibrated velocity comparison	Matched continuation .193381/.199076; calibration features .193407/.199104; Gaussian/Kalman .178545/.192773.	Retain smoother: 7.67%/3.17% vs matched continuation. Improvement in all four platform means, still not large enough on both scopes to claim decisive success.
Acceleration calibration	Physical discrepancy raw .695/.405 to local .263/.175 or pooled .227/.150 m/s ² .	Successful intermediate sensor diagnostic; justified testing complete model. It did not itself establish velocity gain.
Full sensor + bias prior	Continuation .181497/.196584; full correction .181503/.196537; tighter initial bias .181167/.196403, all worse than retained .178545/.192773.	Reject final candidates. Improving physical consistency did not transfer to velocity score.
Bias absorption	Residual bias absorbs 98.5–99.3% of acceleration correction; velocity changes .001 m/s. Initial bias SD .03 gives .175567/.192497; stronger per-state constraints hurt official.	Identified cancellation mechanism, but constraining it is only a small/traded gain. Stop dense bias-strength sweep; retain simpler supported smoother.
Early physical conditioning	Features .181499/.196506; attitude .181486/.196424, essentially matched-control behavior and worse than retained model.	Stop tested warm-start conditioning; not a conclusive result for scratch attitude-aware models.
Recurrent rate	20 Hz .183680/.196428; 20 Hz+memory .183134/.197460.	Raising GRU rate did not fix failures; no promotion or extra continuation. Local decoder already had 20 Hz features.
Gravity-canonical coordinates	Matched .179858/.194217; output coordinates .178760/.195184; input/output .184319/.197850.	Neither beats both matched and retained controls on both sets. Stop this coordinate recipe.
Calibration at encoder input	Corrected gyro .180478/.195209; full correction .181484/.194205.	No consistent control/incumbent gain. Persistent x offsets and underestimation remain; close branch.
Aerodynamic/drag/thrust-plane diagnostics	Drag variations .178625/.192781 or .179702/.192451; thrust-plane settings worse (.189617/.194948 and above).	No two-set promotion; short correlations do not create an absolute-velocity observation or justify stronger physical constraints.
World-motion solver	Best bounded setting .175438/.187323 vs .178545/.192773 (1.74%/2.83%); longer propagation degrades.	CPU prototype, not integrated into portable inference. Below 5% two-set gate, mostly world-like drones; body failures remain. Retain evidence, not an automatic new submission.
Directional covariance and true-gravity diagnostics	Fitting-only full covariance changes prefix body-drone AVE 1.01920/.66806 to 1.00768/.66721; true-gravity changes also small. Label-informed variance oracles can do more.	Weak deployable evidence, insufficient for another covariance/gravity training batch. Oracles are not held-out model scores; general joint calibration remains distinct.
Broader warm/fresh training	Common fresh 24k endpoint .154588/.171648 vs references .167743/.192773. Public .28427, only 1.89% vs preceding eligible entry.	Retain better trained recipe; public gains smaller than validation. Separately selected split peaks are not combined (family-only 16k is not the final schedule). Both holdouts were repeatedly consulted.

Continued on next page

Table VII continued

Study / arms	Results	Decision / limit
Complete scoring segments, first test	.153779/.165595 vs .154588/.171648; long-segment ATE improves 10.20%/8.69%, composite .52%/3.53%; family AVE worse.	Fails prespecified 10% composite/15% long-ATE gates. No refit/upload then. Retain as a limited trajectory-loss ingredient for later controlled combination, not the main velocity solution.
Complementarity/router diagnostics	Uniform eight-prediction mean worsens 6.37%/.72%; label-chosen per-second oracle gains 29%/32%, deployable grouped router only .15%/1.22%. Opposite-half offset/rotation fits often hurt.	Oracle diversity does not imply an implementable mixture. No arbitrary output average, no external-model ensemble submission. Probe shared internal specialization with a matched control.
Privileged geometry teacher	Raw capacity control .154810/.172617; attitude teacher .154921/.172225.	Teacher gate failed; students never trained. Added paths barely used, so the implementation was insufficient evidence to reject geometric learning in general.
Multiscale feature experts	Fixed mixing .158225/.171560; adaptive .158797/.171502 vs .154588/.171648. Routing ablation matters, but adaptive does not reliably beat fixed.	Close exact encoder/router line: human regressions offset drone gains. No evidence of expert collapse because needed usage/loss histories were not retained.
Overnight family-transfer episodes	Episodic control .155078/.169997; transfer .155731/.167904.	Small conflicting gains, not substantial improvement; no refit. The large fitting/holdout gap remains the stronger diagnostic.
Same overnight: signed velocity bins	.158957/.176614, worse on both scopes despite slightly improved fitting score.	Stop this primary decoder; better fitting does not establish better transfer. Distinct from the early auxiliary speed-bin head.
Same overnight: whole-trajectory memory on frozen features	Local correction .154215/.170486; whole memory .154483/.170205.	Near-tie; official gain fragile to removal of one improving family. No substantial winner. Does not disprove end-to-end long-context learning.
Final three-fold combination	Table VI gives the pooled control/segments/speed/both comparison. Segments improves every fold; speed and both lose.	Select segment recipe and replay once on all 475 labels, with fresh sensor/statistics/base. No fold ensemble.
Implementation audit	Checks of axes, units, targets, indices and metrics found no simple implementation error that explained the generalization gap.	The remaining errors motivate modelling and data studies. Reproducibility details are in Appendix B.

F. Implementation Details

Neural architecture. A dilated CNN maps 200 Hz IMU to 192 local features at 20 Hz. Two 64-dimensional branches encode 46 per-second statistics and three-second spectrogram neighborhoods (128-sample Hann window, hop 50). Pooling gives a 4 Hz input to a two-layer bidirectional GRU with 256 units per direction; its state and local features feed a 20 Hz decoder. Spectral neighborhoods are clamped to context boundaries. A shared velocity head and four internal residual heads form

$$u_t = h_0(z_t) + \sum_{k=1}^4 \pi_{t,k} h_k(z_t), \quad \pi_t = \text{softmax}(g_t). \quad (2)$$

Platform supervision trains the gate, but the inference gate is computed from IMU features. The four heads are trained within the same shared model; there is no dispatch to independently trained platform networks.

Smoothing objective. At one-second boundaries, the state is $x_j = [v_j^\top, b_j^\top, g_j^\top]^\top \in \mathbb{R}^9$, containing body velocity, accelerometer bias and gravity. Raw acceleration and corrected gyro define transition matrices F_j , offsets d_j , and window-mean observation operators H_j, c_j . Within each context, we solve

$$\begin{aligned} x^* = \arg \min_x & \sum_j \|H_j x_j + c_j - \bar{u}_j\|_{R_j^{-1}}^2 \\ & + \sum_j \|G x_j - \hat{g}_j\|_{S_j^{-1}}^2 \\ & + \sum_j \|x_{j+1} - F_j x_j - d_j\|_{Q_j^{-1}}^2 + \|x_0\|_{P_0^{-1}}^2. \end{aligned} \quad (3)$$

Here G selects gravity, $\bar{u}_j$ is the neural one-second mean, and $\hat{g}_j$ is learned gravity. The initial velocity precision is 10^{-6} , avoiding a known-zero initial-speed assumption; initial bias standard deviation is 1 m/s^2 . The normal equations are block tridiagonal with 9×9 blocks, solved by parallel cyclic reduction in double precision. This is Gaussian smoothing conditional on the learned operators, not a full nonlinear error-state EKF.

The decoder predicts a velocity scale, three process scales and a blending strength. Fifteen physical features also supply a confidence-gated decoder correction. Dense smoother velocity s_t is combined with neural velocity as

$$\hat{v}_t = (1 - \alpha_j)u_t + \alpha_j s_t, \quad \alpha_j = \sigma(a_j) \overline{p_{\text{body}_j}}. \quad (4)$$

Because both paths use the same IMU, their errors are correlated. The learned scales are effective fusion weights; we do not claim independent observations or statistically calibrated posterior uncertainty.

Input and target processing. Acceleration is divided by 9.81 and gyro by π . Each 20 Hz velocity target averages ten released samples; each one-second target averages 200. We do not rotate a mean world-frame velocity at a single attitude to construct body-frame targets. Padding is masked. The sensor timing mixture uses offsets $\{0, -5, -10, -20, +5\}$ samples. Physical features use raw acceleration and corrected gyro.

Optimization and segment supervision. Velocity training uses AdamW, batch size 32, initial learning rate 3×10^{-4} , weight decay 10^{-4} , 5% warmup, cosine decay, gradient clipping at 1 and EMA decay 0.995. Continuations reset AdamW and start at 10^{-4} . Sensor fitting uses batch size 16, eight-second crops, AdamW with weight decay 10^{-3} , and a cosine schedule from 3×10^{-4} to 10^{-5} .

The ordinary crop objective combines smooth one-second vector error, dense Huber error, integrated errors over 1/4/16/32-second horizons and gate cross-entropy. Their dense/integration/gate weights are 0.25/0.1/0.02. Calibration adds a raw-velocity anchor (0.1) and an effective uncertainty loss (0.002). Ground-truth geometry enters these fitting losses, never the inference interface.

Complete-segment supervision differentiates through the actual 64-second, stride-32 inference windows and their overlap weighting. A 20-meter scoring segment may span more than one context; every intersecting context contributes, including the final boundary-anchored context. One full segment is sampled per platform, with trajectories and then segments selected uniformly. Predicted world displacements use fitting-side orientation and the official integration convention. A no-scale SE(3) alignment minimizes trajectory residuals; the optimal alignment is detached when differentiating the minimized loss to avoid unstable SVD derivatives. The segment coefficient ramps during the first quarter of continuation to

$$\lambda_{\text{seg}} = \frac{0.4/3.1160277267}{0.6/0.7356384388}. \quad (5)$$

This preserves the score's relative velocity/path weighting while retaining the ordinary crop losses.

Inference details. Overlapping contexts use weights $0.1 + \sin^2(\pi(i + 0.5)/n)$ at dense output index i in a context with n outputs. There is no rotation test-time augmentation (TTA), no fitting on test data and no state shared across trajectories. Inference uses BF16 for selected neural operations, float32 for recurrence and geometry, and float64 for the Gaussian solve.